

Operating Systems — Process Scheduling

2.1 What is a Process?

A process is defined as a program in execution, including the program code, its current activity represented by the value of the program counter, and the contents of the processor registers. A process is distinct from a program: a program is a passive entity stored on disk, while a process is an active entity with a defined life cycle.

Every process moves through a series of states during its lifetime: new, ready, running, waiting, and terminated. The new state occurs when the process is being created, the ready state occurs when the process is waiting to be assigned to a processor, the running state occurs when instructions are being executed, the waiting state occurs when the process is waiting for some event such as I/O completion, and the terminated state occurs when the process has finished execution.

The process control block, commonly abbreviated as PCB, is a data structure maintained by the operating system for every process. It contains information such as the process state, program counter, CPU registers, CPU scheduling information, memory management information, and I/O status information.

2.2 CPU Scheduling Fundamentals

CPU scheduling is defined as the process by which the operating system decides which process in the ready queue is allocated the CPU next. The goal of a good scheduling algorithm is to maximize CPU utilization and throughput while minimizing turnaround time, waiting time, and response time.

Turnaround time is defined as the total time taken from process submission to process completion. Waiting time is defined as the total time a process spends waiting in the ready queue. Response time is defined as the time from process submission until the first response is produced, which is particularly important for interactive systems.

2.3 Scheduling Algorithms

First-Come First-Served, abbreviated FCFS, is the simplest scheduling algorithm: processes are executed in the order they arrive in the ready queue. Although simple to implement, FCFS can lead to the convoy effect, where a long process delays all subsequent shorter processes.

Shortest Job First, abbreviated SJF, selects the process with the smallest estimated execution time next. SJF is provably optimal in terms of minimizing average waiting time, but it requires knowledge of the length of the next CPU burst, which is often difficult to predict in practice.

Round Robin scheduling assigns each process a fixed time slice, called a time quantum, and cycles through all processes in the ready queue. Round Robin is particularly well suited to time-sharing systems because it guarantees that every process receives a fair share of the CPU within a bounded

amount of time.

Priority scheduling assigns a priority number to each process, and the CPU is allocated to the process with the highest priority. A significant problem with priority scheduling is starvation, where a low-priority process may never execute; this is typically solved using a technique called aging, which gradually increases the priority of processes that wait for a long time.

2.4 Multilevel Queue and Feedback Scheduling

A multilevel queue scheduling algorithm partitions the ready queue into several separate queues based on process characteristics, such as foreground versus background processes, with each queue having its own scheduling algorithm. This approach is often difficult topics for students because it requires understanding how multiple algorithms interact simultaneously.

Multilevel feedback queue scheduling extends the multilevel queue approach by allowing processes to move between queues based on their behavior and CPU burst characteristics, providing greater flexibility than a static multilevel queue.

2.5 Chapter Summary and Review Questions

In summary, a process moves through the states new, ready, running, waiting, and terminated during its lifetime, and the operating system uses scheduling algorithms such as FCFS, SJF, Round Robin, and Priority Scheduling to determine which process receives the CPU next. Each algorithm makes different tradeoffs between simplicity, fairness, and average waiting time.

Review question: What is the difference between turnaround time and waiting time? Review question: Why is SJF considered optimal, and what practical limitation prevents its widespread use? Review question: How does aging solve the starvation problem in priority scheduling?